The Transform Calculus: Birth-Cut Constraint Multiset Rewriting, the Certain-Check Proof Logic, and the Tier-0 `tcalc` Kernel for MeTTa

Formalized in Lean 4

(`Mettapedia.Languages.ProcessCalculi.ConstraintMultisetRewriting`)

November 8, 2026

Abstract

When a rewriting or logic engine combines multiset state, unification variables, and negative or cardinality guards (such as testing the absence of a pattern or counting its matches), operational engines routinely suffer from four pathologies: testing absence only at a principal unifier is unsound under later variable instantiation; collapsing spaces into sets destroys occurrence multiplicity while naive multiset counting explodes combinatorially; inspecting raw syntactic terms leaks across equated variables; and conflating symbolic consistency with closed inhabitation allows rules guarded by uninhabited types to fire vacuously.

We present the *Transform Calculus*, a formal model of transactional constraint multiset rewriting designed as the semantic kernel for MeTTa and mechanized in Lean 4 (over 270 sorry-free modules). Rather than a monolithic machine, the calculus is structured around a single categorical step interface (`Interface.Frames` and `Interface.Step`) whose admission law (`admits_iff`) separates physical occurrence commits from semantic constraint solving and universal model-theoretic guard entailment at the firing's birth cut. Its primary first-order instance (`AnnotatedRuns.firstOrder` and `Spaces.located`) is governed by the *certain-check proof calculus* (`CertainChecks.Derives`), a three-rule inference system that decides equality, absence, and exact cardinality over every solution of an equation store by refuting candidate extensions via occurs-checked unification. We prove soundness and completeness of `CertainChecks.Derives` for free Henkin models and the free term algebra (`derives_iff_termModel`), establish the incompleteness of the ground Herbrand model (`oneConstant_separates`), and characterize query-free firing admission as formal derivability (`admitsDerivably_iff`). We then show how SLD resolution, guarded superposition (via upward guard closure `entails_closure_iff`), ground Knuth–Bendix completion, MeTTa space operations (`match`, `collapse`, equation calls, and MM2 `exec`), and higher-order contextual and dependent instances inhabit the same step interface. Finally, we present `tcalc`, a 1,630-line C99 Tier-0 reference kernel implementing the Transform Calculus step interface, its modular prepared-step and candidate-cursor bridge to CeTTa (`TcPreparedStep`, `TcAdmissionReceipt`, `TcCandidateCursor`), and five foundational proof verifiers built directly as `tcalc` rule systems: Metamath (`micromath_tcalc`), Metamath Zero (`mm0_tcalc`/`mmb_tcalc`), HOL Light (`hol_tcalc`), Megalodon across all four foundational profiles (`mg_tcalc`: Egal/HoTTG, HF, Mizar, HOAS), and Dedukti's $\lambda\Pi$-calculus modulo rewriting (`dk_tcalc`).

Contents

1 Introduction: Four Pathologies of Open Rewriting **3**

2 The Step Interface and Its Instance Architecture **4**
2.1 Frames, Solutions, and Principal Answers (`Interface.Frames`) 5
2.2 The Step Signature (`Interface.Step`) and the Instance Matrix 5

3 MeTTa Surface Operations and Located Spaces **8**
3.1 Translation of MeTTa Operations to the Step 8

4 The Certain-Check Proof Calculus **10**
4.1 Syntax, Regions, and Inference Rules of `CertainChecks.Derives` 10
4.2 Worked Example: Absence Under Later Instantiation 10
4.3 Main Theorems of the Proof Logic (with Proof Sketches) 12
4.4 Conflicting Evidence vs. Inconsistent Stores and the 4×4 Separation Matrix . . 13

5 Proof Calculi and Fair Search as Firings **14**
5.1 Resolution, Guarded Superposition, and Ground Completion 14
5.2 The Search Outcome Algebra, Guard Polarities, and Fair Exploration (`Step.Search*`, `Step.GuardFragments*`, `Step.FairSearch*`) 15

6 Programs on the Step: Compilation, Interpretation, and Binders **16**

7 Concurrency, Races, and Behavioral Equivalence **17**

8 Contextual & Dependent Instances and Formalization Status **18**
8.1 Contextual and Dependent Instances and the Open/Closed Admission Trap . . . 18
8.2 The η-Long Fibre and the Three Fibre-Join Theorems 18
8.3 Honest Status of Dependent Unification 19

9 Comparison with Related Calculi and Systems **20**

10 The C Runtime, Proof Checkers, and Verification Boundaries **21**
10.1 Preparation, admission and the host boundary 22
10.2 Proof-system implementations and their residual trust 22
10.3 Translation and machine-code verification 23

11 Conclusion and Mechanization Summary **23**

1 Introduction: Four Pathologies of Open Rewriting

Languages that combine pattern-directed multiset rewriting, logic variables, and reflective queries over a shared workspace—including MeTTa [15], Constraint Handling Rules (CHR) [13], Maude [22], and the ρ-calculus [21]—face a recurring tension between operational convenience and logical soundness. In mechanized stress-testing of candidate operational semantics in Lean 4, four foundational pathologies repeatedly emerge whenever open variables interact with workspace queries:

P1. The Principal-Answer Fallacy for Absence. Suppose a workspace contains the open atom `(p $X)` at occurrence o_1, the equation store K is empty, and a default rule asks for the *absence* of `(p a)` before emitting `(fallback a)`. Evaluated at the most general unifier $\mu = \mathrm{id}$ of K, the terms $\mu(\texttt{(p a)}) = \texttt{(p a)}$ and $\mu(\texttt{(p \$X)}) = \texttt{(p \$X)}$ are syntactically distinct. An engine that tests negation-as-absence only at the current principal unifier fires the default rule immediately. Yet a subsequent step may bind $\$X \doteq \texttt{a}$, retroactively falsifying the absence premise in the refined model. As happens in free-variable tableaux [12, 16], an existential variable in the state turns a negative guard into a universal closure obligation over *all* solutions of K.

P2. Set Collapse versus Combinatorial Count Explosion. Suppose a rule requires exactly n occurrences matching pattern p in the workspace. If the workspace is a set (as in MM2/MORK [15]), duplicate physical witnesses of `(coin head)` collapse into one. Conversely, in a multiset with open variables, checking that $|\{o \in W \mid \sigma(p) = \sigma(\ell(o))\}| = n$ holds across *every* solution σ of K appears to require enumerating all subsets of W of size n. How can an engine decide universal exact cardinality in a single pass without combinatorial subset enumeration?

P3. Syntactic Query Leakage. If a workspace query inspects the raw syntax of stored labels rather than their equivalence classes under solutions of K, two configurations with identical solution sets can be separated by an observation (`ModuloControls.syntactic_region_separates`). Sound observations must read through solutions (`QueriesReadThroughSolutions`).

P4. The Uninhabited-Type Admission Trap. In a typed or proof-carrying workspace where a fresh metavariable $\$h : A$ represents a proof obligation, symbolic solvability (`Frames.Solvable`) asks whether K has a solution into *some* later declaration frame. Because the extended frame $\Gamma, (\$h : A)$ maps to itself by the identity substitution regardless of whether A has any closed inhabitants (`solvable_extend_iff`), symbolic admission permits firings guarded by empty types (`obligation_trap`, `trap_helper`, and `Step.TypingContractsControls.void_trap`). Separating symbolic consistency from closed inhabitation (`ClosedSolvable`, `ClosedAdmits`) is essential when firings represent proof steps.

Contributions and Architecture. Rather than inventing an ad-hoc operational patch for each pathology, the *Transform Calculus* resolves them through a layered architecture mechanized in Lean 4 under `Mettapedia.Languages.ProcessCalculi.ConstraintMultisetRewriting`:

- **One Categorical Step Interface, Multiple Exact Instances (§2).** We define a category of declaration frames acting on terms (`Interface.Frames`) and a single step signature (`Interface.Step`) whose admission theorem (`admits_iff`) requires physical occurrence enabling, semantic solvability of the firing's information store, and universal entailment (`Frames.Entails`) of all guards and queries across every solution at the firing's birth cut. Table 1 maps each Lean instance to its exact features, avoiding any conflation between first-order, located, disequality, contextual, and dependent instances.

- **MeTTa Operation Compilation and Located Spaces (§3).** We give an explicit translation table (Table 2) relating MeTTa surface constructs—`match`, `collapse`, stored equations `(= (f $x) body)`, `add-atom`/`remove-atom`, and MM2 `exec`—to firings and store-level observations (`MeTTaOperationsMatch`, `MeTTaOperationsCollapse`, `MeTTaOperationsCall`, `MeTTaOperationsExec`), together with handle-located spaces (`Step.Spaces`) and per-handle facet permissions (`Step.Permissions`).
- **The Certain-Check Proof Calculus (§4).** The centerpiece of the first-order development is `CertainChecks.Derives` (`Step/ModelDerivation.lean`), a proof calculus with three inference rules (Der-Equal, Der-Absent, Der-Count) that reduces universal birth-cut entailment to occurs-checked unification (`unifyTotal`) and pairwise compatibility refutation (`FirstOrderChecks.compatible`). We trace a complete worked MeTTa example of absence under later instantiation and provide proof sketches for:
  (i) the *Absence Refutation and Count Squeeze Theorem* (`entails_absent_iff`, `absent_iff_possible_empty`, `Frames.Principal.entails_count_iff`, `count_iff_regions`);
  (ii) the *Henkin Completeness and Herbrand Incompleteness Theorem* (`derives_sound_free`, `derives_complete_generic`, `derives_iff_termModel`, `oneConstant_separates`, `admitsDerivably_iff`); and
  (iii) the 4×4 *Evidence Separation Matrix* (`Evidence.Family.profiles_distinct`) over the disequality step (`Disequality.step`).
- **Automated Reasoning, Programs, Concurrency, and Dependent Status (§5–§9).** We show how first-order resolution (`Step.ResolutionFiring`), guarded superposition via upward closure (`entails_closure_iff`), ground Knuth–Bendix completion (`Step.CompletionCorrectness`), fair search (`Step.SearchPolicy`), MeTTa program compilation and interpretation (`Step.Program*`), concurrency diamonds and Milner's race spectrum (`Step.RacePair`, `Step.RaceSpectrum`), and the contextual/dependent instances (`Step.DependentFull`, `Step.DependentClosed`, `Step.DependentFullFibre`) fit together, accompanied by a transparent accounting of proved theorems versus open problems (§8).
- **The Tier-0 C Kernel (`tcalc`), CeTTa Cursor Bridge, and Five-Foundation Verifier Suite (§10).** We describe the 1,630-line C99 Tier-0 reference kernel (`tcalc`), its modular prepared-step and candidate-cursor interface (`TcPreparedStep`, `TcAdmissionReceipt`, `TcCandidateCursor`), its schema compiler (`tcalc_compile`) and flat x86-64 leaf (`flat_fire`), and five foundational proof verifiers hosted as `tcalc` rule systems: Metamath, Metamath Zero, HOL Light [17], Megalodon [6], and Dedukti ($\lambda\Pi$-calculus modulo rewriting) [10, 4]. We analyze the 8-rule structural duality between Simple Type Theory with prenex polymorphism (`mg_tcalc`) and Dependent Type Theory modulo rewriting (`dk_tcalc`), and show how `coGSLT.module` and executable OSLF connect them functorially.

2 The Step Interface and Its Instance Architecture

A central design principle of the Lean 4 formalization is that capabilities such as multiple spaces, handle permissions, disequality constraints, higher-order binders, and dependent telescopes are *not* piled into a single monolithic record. Instead, `Step/Interface.lean` defines a single categorical signature once, and each capability is constructed as an explicit instance or combinator over that interface.

2.1 Frames, Solutions, and Principal Answers (`Interface.Frames`)

Definition 2.1 (Category of Frames Acting on Terms — `Interface.Frames`)**.** A frame system $\mathcal{X}$: `Frames` Tm Hom over a type of frame objects Obj consists of:

1. For each frame a : Obj, a type of terms $\mathrm{Tm}(a)$ equipped with an equivalence relation $\equiv_a$ (`X.E`, `X.equivalence`).
2. For each pair of frames a, b : Obj, a type of frame morphisms $\mathrm{Hom}(a, b)$ with identity $\mathrm{id}_a : \mathrm{Hom}(a, a)$ and composition $g \circ f : \mathrm{Hom}(a, c)$ for $f : \mathrm{Hom}(a, b)$ and $g : \mathrm{Hom}(b, c)$.
3. A capture-avoiding instantiation action $\mathrm{act} : \mathrm{Hom}(a, b) \to \mathrm{Tm}(a) \to \mathrm{Tm}(b)$ satisfying the functorial action laws $\mathrm{act}(\mathrm{id}_a, t) = t$ (`act_id`), $\mathrm{act}(g \circ f, t) = \mathrm{act}(g, \mathrm{act}(f, t))$ (`act_comp`), and congruence $s \equiv_a t \implies \mathrm{act}(f, s) \equiv_b \mathrm{act}(f, t)$ (`preserves`).

An *equation store* K at frame a (`EquationStore`) is a finite list of formal equations $s \doteq t$ with $s, t : \mathrm{Tm}(a)$. Throughout this paper, we write $s \doteq t$ for equations in the constraint store K, reserving `(= lhs rhs)` for MeTTa program-equation atoms stored as data in a workspace.

Definition 2.2 (Models, Solvability, Entailment, and Principal Answers)**.** Let $\mathcal{X}$ be a `Frames` structure and K an equation store at frame a:

- **Model** (`Frames.Models`): A morphism $\sigma : \mathrm{Hom}(a, b)$ into a later frame b *models* K, written $\sigma \models K$, iff $\mathrm{act}(\sigma, s) \equiv_b \mathrm{act}(\sigma, t)$ for every $(s \doteq t) \in K$.
- **Semantic Solvability** (`Frames.Solvable`): K is solvable iff $\exists (b : \mathrm{Obj})\ (\sigma : \mathrm{Hom}(a, b)),\ \sigma \models K$.
- **Universal Ask / Entailment** (`Frames.Entails`): Given a family of observations $\Phi : (b : \mathrm{Obj}) \to \mathrm{Hom}(a, b) \to \mathrm{Prop}$, K entails Φ (written $K \models \Phi$) iff $\forall (b : \mathrm{Obj})\ (\sigma : \mathrm{Hom}(a, b)),\ \sigma \models K \implies \Phi(b, \sigma)$.
- **Principal Answer** (`Frames.Principal`): A morphism $\mu : \mathrm{Hom}(a, m)$ is *principal* for K iff $\mu \models K$ (`sound`) and for every b and $\sigma : \mathrm{Hom}(a, b)$ with $\sigma \models K$, there exists a residual morphism $\rho : \mathrm{Hom}(m, b)$ such that $\forall t : \mathrm{Tm}(a),\ \mathrm{act}(\sigma, t) \equiv_b \mathrm{act}(\rho \circ \mu, t)$ (`factors`).

Every classical substitution monoid $\mathcal{A}$: `SubstitutionAction` T S embeds as the one-object frame category `Frames.ofSubstitution` $\mathcal{A}$ over Obj = **1** (`PUnit`), where `Frames.Solvable`, `Frames.Entails`, and `Frames.Principal` coincide definitionally with $\mathcal{A}$`.Consistent`, $\mathcal{A}$`.Entails`, and $\mathcal{A}$`.Principal` (`ofSubstitution_solvable_iff`, `ofSubstitution_entails_iff`, `ofSubstitution_principal_iff`).

2.2 The Step Signature (`Interface.Step`) and the Instance Matrix

Definition 2.3 (The Step Interface — `Interface.Step`)**.** Over a frame system $\mathcal{X}$: `Frames` Tm Hom and a decidable type of physical occurrence identities O, an instance P : `Interface.Step` $\mathcal{X}$ O State Firing specifies:

1. For each state s : State, its declaration frame $\mathrm{frame}(s)$: Obj, its current equation store $\mathrm{knowledge}(s)$: `EquationStore`$(\mathrm{Tm}(\mathrm{frame}(s)))$, and its finite set of alive occurrence identities $\mathrm{present}(s)$: `Finset` O.
2. For each candidate firing f : Firing(s), its physical occurrence selection $\mathrm{selection}(f) = \langle \mathrm{taken}, \mathrm{read}, \mathrm{made} \rangle$: `Selection` O, its target frame $\mathrm{target}(f)$: Obj reached via $\mathrm{extension}(f) : \mathrm{Hom}(\mathrm{frame}(s), \mathrm{target}(f))$, and its birth-cut equation store $\mathrm{information}(f)$: `EquationStore`$(\mathrm{Tm}(\mathrm{target}(f)))$, which must contain the transported prior knowledge (`transported`).
3. A well-formedness predicate $\mathrm{Shapes}(f)$: Prop, a birth-cut observation predicate $\mathrm{Holds}(f, b, \sigma)$: Prop for each solution $\sigma : \mathrm{Hom}(\mathrm{target}(f), b)$, and an admission predicate $\mathrm{Admits}(f)$: Prop characterized by the universal law (`admits_iff`):

$$\begin{aligned} P.\mathrm{Admits}(f) \iff\ & P.\mathrm{Shapes}(f) \\ \wedge\ & \mathrm{Enabled}(P.\mathrm{present}(s),\ P.\mathrm{selection}(f)) \\ \wedge\ & \mathcal{X}.\mathrm{Solvable}(P.\mathrm{information}(f)) \\ \wedge\ & \mathcal{X}.\mathrm{Entails}(P.\mathrm{information}(f),\ P.\mathrm{Holds}(f)). \end{aligned} \tag{1}$$

4. A transition function $\mathrm{result}(f, h_{\mathrm{adm}})$: State and a settling morphism $\mathrm{settle}(f, h_{\mathrm{adm}})$: $\mathrm{Hom}(\mathrm{target}(f), \mathrm{frame}(\mathrm{result}(f, h_{\mathrm{adm}})))$ updating the alive occurrences by multiset commit (`present_result`: $(P.\mathrm{present}(s) \setminus \mathrm{taken}) \cup \mathrm{made}$ when $\mathrm{taken}, \mathrm{read} \subseteq P.\mathrm{present}(s)$, $\mathrm{taken} \cap \mathrm{read} = \emptyset$, and $\mathrm{made} \cap P.\mathrm{present}(s) = \emptyset$) and transporting the knowledge along settle (`knowledge_result`).

Runs of any instance P are dependent words of admitted firings (`Interface.Step.Run`). Just as the π-calculus and ρ-calculus are governed by a single reaction rule (Comm) modulo structural congruence, the entire Transform Calculus is governed by a single firing rule [Fire] over $\mathcal{X}$ (`admits_iff`, `present_result`, `knowledge_result`):

$$\frac{P.\mathrm{Shapes}(f) \quad \mathrm{Enabled}(W, (T, R, M)) \quad \mathcal{X}.\mathrm{Solvable}(\Gamma', I_f) \quad \mathcal{X}.\mathrm{Entails}(\Gamma', I_f, P.\mathrm{Holds}(f))}{\langle \Gamma \,;\, K \,;\, W \rangle \xrightarrow{f} \langle \Gamma'' \,;\, \mathrm{act}(\mathrm{settle}(f), I_f) \,;\, (W \setminus T) \cup M \rangle} \;[\textsc{Fire}]$$

where $\Gamma = P.\mathrm{frame}(s)$, $K = P.\mathrm{knowledge}(s) \subseteq I_f$, $W = P.\mathrm{present}(s)$, and $\Gamma \xrightarrow{\mathrm{extension}(f)} \Gamma' \xrightarrow{\mathrm{settle}(f)} \Gamma''$. Around [Fire] sit three structural components: (i) *structural congruence* $\equiv$, comprising occurrence renaming (`Iso`), observation equivalence modulo solutions of K (`Step.ObservationModulo`, `QueriesReadThroughSolutions`), and garbage-collection of unused unknowns (`NativeUpToDead`, `ContextualDependentEtaLongWeakening`); (ii) *the choice of frame category* $\mathcal{X}$, which determines what the unknowns are (first-order variables, disequation frames, higher-order contextual patterns (`$M x`), or dependent $\Pi/\Sigma/\mathrm{Id}$ telescopes); and (iii) *derived forms*, such as located spaces (`Spaces`) and compiled program schemas (`ProgramEquations.compile`). Along any run $r : P.\mathrm{Run}(s)$, knowledge only refines: every solution of the final state's knowledge pulls back along r.toLast to a solution of the initial state's knowledge (`Step.Run.models_start`, `Step.Run.solvable_start`), and any state with unsolvable knowledge admits no firing of any instance (`Evidence.not_admits_of_not_solvable`). Runs form a free category under concatenation (`Step.Run.append`), and monoid-valued site accounts are functors out of it (`Step.Run.account_append`).

How the Instances Relate (Table 1). Table 1 records the exact scope of each instance in the repository:

- `AnnotatedRuns.firstOrder` is the one-frame instance over any `SubstitutionAction` $\mathcal{A}$. A configuration $C = \langle K, W \rangle$ carries an equation store K and a present set W : `Finset` O, with an ambient labelling $\ell : O \to T$. A firing $f = \langle r, Q, \mathrm{sel} \rangle$ combines a five-port `RuleInstance` $r = \langle \mathrm{take}, \mathrm{read}, \mathrm{tell}, \mathrm{checks}, \mathrm{produce} \rangle$, a list of predicate queries Q, and a selection $\mathrm{sel} = \langle \vec{o}_{\mathrm{take}}, \vec{o}_{\mathrm{read}}, \vec{o}_{\mathrm{make}} \rangle$. Its birth-cut store (`Selection.problem`) appends to K the matching equations $\mathrm{take}_i \doteq \ell(o_{\mathrm{take},i})$, $\mathrm{read}_j \doteq \ell(o_{\mathrm{read},j})$, and $r.\mathrm{tell}$.
- `Disequality.step` adds told disequations $s \neq t$ without modifying `Interface.Step` by placing the disequation store D into the *frame object* (`Disequality.Frame`) and restricting frame morphisms to substitutions that pull separating solutions back to separating solutions (`Disequality.frames`).
- `DependentFull.step` works over the typed $\beta\eta$ frames of the dependent unifier (`ContextualDependentEta.frames`), with helper telescopes, typed takes/reads/makes, heterogeneous equation tells/guards, and solution-respecting queries (`DependentFull.Query`). Crucially, in `DependentFull.step`, the post-state f.after remains at the extended telescope $\Delta = f.\mathrm{helpers.frame}$ and its settlement morphism is the identity $\mathbf{1}_\Delta$—there is no implicit variable-elimination settlement step in `DependentFull.Firing.after`.

Lifting Features to the Interface and Exact Restrictions (`Step.InterfaceTable`, `Step.DisequalityFrames*`, `Step.EvidenceInterface*`, `Step.PermissionsInterface*`, `Step.SpacesInterface`). Rather than leaving capabilities tied to individual instances, the formalization collects the full feature × instance correspondence in `Step.InterfaceTable` (`InterfaceTable.cut_preserved`) and

Table 1: The Instance Matrix of `Interface.Step`: each capability in the formalization is a specific instance or combinator over `Interface.Step`, never a conflated mega-record.

Instance / Combinator	**Lean Module**	**Frames** (Obj, Hom)	**Guards & Queries**	**Settlement** settle
First-order step `AnnotatedRuns.firstOrder`	`AnnotatedRuns`	One frame **1** (`ofSubstitution`)	`Check`: `equal`, `absent`, `count` + census `Query`	Identity $\mathrm{id}_{\mathbf{1}}$
Guarded FO step `Superposition.guarded`	`Step.SuperpositionGuarded`	One frame **1** (`ofSubstitution`)	FO checks + solution predicates $G : S \to \mathrm{Prop}$	Identity $\mathrm{id}_{\mathbf{1}}$
Located spaces `Spaces.located`	`Step.Spaces`	One frame **1** over rigid handle pairs (h, t)	Space-scoped checks `checkAt` & queries at h	Identity $\mathrm{id}_{\mathbf{1}}$
Permission restriction `Permissions.permitted`	`Step.Permissions`	Inherited from base step P (`Spaces.restrict`)	Restricts Shapes by `FormedUnder` $(\Gamma(h, \mathrm{facet}))$	Inherited from P
Disequality step `Disequality.step`	`Step.Disequality`	Disequation stores D (`Disequality.frames`)	FO checks & queries over D-separating solutions	Identity on $D \cup D_{\mathrm{told}}$
Contextual HO step `Contextual.step`	`Step.Interface` `Step.ContextualDiamond`	Declaration frames (Names, arity) (`contextual`)	Equations & queries on `Observation`	Identity on extended frame
Dependent helper step `Dependent.helperStep`	`Step.DependentHelpers` `Step.TypingStates`	Formed contexts Γ (`Dependent.frames`)	Fresh helper $x : A$ + equation guards	Identity on $\Gamma, (x : A)$
Dependent full step `DependentFull.step`	`Step.DependentFull`	Typed $\beta\eta$ contexts (`ContextualDependentFrames`)	Helper telescope Δ, Eqns, reads, makes, tells, `Query`	Identity $\mathbf{1}_\Delta$ on $\Delta =$ f.helpers.frame
Closed admission `Step.ClosedAdmits`	`Step.DependentClosed`	Any $\mathcal{X}$ with designated closed frame point	Uses `ClosedSolvable` and `ClosedEntails`	Modifier on admission

lifts features directly to `Interface.Frames` and `Interface.Step`, accompanied by separating counterexamples whenever a checkable restriction is required:

1. **Disequation Frames Over Any $\mathcal{X}$: `Interface.Frames` (`Step.DisequalityFrames`, `Step.DisequalityFramesStep`, `Step.DisequalityFramesRestriction`, `Step.DisequalityFramesUnguarded`):** `DisequalityFrames.frames` equips any frame category $\mathcal{X}$ with disequation stores separated by frame morphisms (`Separates`, `separates_id_iff`), recovering `Disequality` at $\mathcal{X} =$ `ofSubstitution` $\mathcal{A}$ (`solvable_iff_old`). Lifting a prestep to carry told disequations (`Prestep.withDisequations`) requires that settlement reflect equality (`Prestep.Reflects`); without reflection, a prestep that settles by guessing $X \mapsto 0$ admits $X \doteq 1 \wedge X \neq 0$ and transports the disequation to $0 \neq 0$, which no frame contains (`guess_breaks_frame`, `guess_not_reflects`). For non-presteps whose result depends on base admission (such as the native step), `Unguarded.withDisequations` lifts unguarded firings whenever settlement reflects equality (`Unguarded.native_unguarded`).
2. **Interface-Level Run Consistency and Dependent 4×4 Evidence Table (`Step.EvidenceInterface`, `Step.EvidenceInterfaceFamily`, `Step.EvidenceInterfaceNative`):** Any instance whose settlement keeps solvable stores solvable (`SettlesConservatively`, for which `RetractsSettlement` suffices via `conservative_of_retracts`) preserves knowledge solvability along every run (`run_solvable`, instantiated at `firstOrder_retracts`, `dependent_retracts`, `helper_retracts`, `withDisequations_retracts`, and on the non-retracting native step via `Native.native_conservative`), whereas a guessed settlement enters an unsolvable store (`guess_enters_inconsistency`). Moreover, `EvidenceInterface.DependentFamily` lifts the 4×4 evi-

dence matrix (Table 3) to `DisequalityFrames.Dependent.withDisequations` at frame $X : U_0, x : X, g :$ Ground (`profiles_distinct`).

3. **Interface-Level Permissions and Space Independence (`Step.PermissionsInterface`, `Step.PermissionsInterfaceControls`, `Step.SpacesInterface`):** Existential read stability holds on *every* instance of `Interface.Step` (`PermissionsInterface.witness_present`), while universal absence preservation across firings without `send` holds on every instance satisfying `LabelLaw` (`absence_preserved`, instantiated at `locatedLabels` and `dependent_absence_preserved`) and fails if a firing relabels a surviving occurrence in place (`relabel_spoils_absence`). Similarly, `SpacesInterface` proves `Transport.Independent` for quiet firings at distinct spaces on both the located first-order step (`located_independent`) and the dependent full step (`dependent_independent`), alongside the dependent shared-variable counterexample (`dependent_shared_variable_crosses_spaces`).

The State Triple $\langle \Gamma ; K ; W \rangle$, Three Kinds of Variables, and Native LCF Proof Kernels. Separating a configuration into $\langle \Gamma ; K ; W \rangle$ resolves two longstanding tensions in MeTTa:

- **Three Distinct Variable Roles:** The calculus cleanly separates (1) *lexical bound variables* (λx, `let`, template variables of `map`/`fold`), which are α-renamed and never unified; (2) *frame unknowns* (Γ, MeTTa's $\$x$), which persist in K across firings for two-way unification; and (3) *rule holes*, which are freshened per firing via extension$(f) : \Gamma \to \Gamma'$. Under binders, Miller pattern raising `($M x)` explicitly lists the bound variables on which an unknown $\$M$ may depend, preventing variable capture while allowing broad unification outside the scope (`Step.ProgramBinders*`).
- `Frames` **as a Native Proof Kernel:** Under the propositions-as-types reading of `Interface.Frames`, unknowns in Γ are open proof obligations, a model $\sigma \models K$ is a proof witnessing them, a firing f is an inference step, and `Step.Run.models_start` composes proofs along a run by pulling any final solution back to the initial obligations, while space permissions (`PermissionsInterface`) enforce LCF theorem sealing without an external seal primitive (only kernel schemas hold `send` on the theorem space).

3 MeTTa Surface Operations and Located Spaces

To connect the Transform Calculus directly to MeTTa programming, the formalization provides an explicit semantics for MeTTa's workspace operations (`MeTTaOperationsMatch`, `MeTTaOperationsCollapse`, `MeTTaOperationsCall`, `MeTTaOperationsExec`) and for multi-space handles and capabilities (`Step.Spaces`, `Step.Permissions`).

3.1 Translation of MeTTa Operations to the Step

In MeTTa, a workspace (or *space*, denoted `&self` for the default space) holds S-expression atoms, which may be ground data `(edge a b)`, open data `(p $z)`, typing assertions `(: x Int)`, or program equations `(= (f $x) (g $x))`. Table 2 summarizes how each core MeTTa operation translates into the Transform Calculus and cites the corresponding adequacy and obstruction theorems.

Three foundational lessons from `MeTTaOperations*` merit emphasis:

1. **Multiplicity and Per-Use Opening in `match` (`MeTTaOperationsMatch`).** Encoding `(match` s p t`)` as the family of valid single-read firings indexed by $o \in W$ reproduces the reference answer bag with exact multiplicity (`encodedAnswers_eq_reference`): on `(p 1) (p 1) (p 2)` with pattern `(p $x)` and template `(r $x)`, the encoded answer bag is `(r 1) (r 1) (r 2)` (`probe_answers`). However, if a stored atom `(q $z)` has a live variable in the shared store, reading it directly binds $\$z$ globally and blocks a subsequent query `(q 7)` after `(q 5)`

Table 2: Translation of MeTTa surface operations into the Transform Calculus (`MeTTaOperations*`).

MeTTa Operation	**Transform Calculus Realization**	**Key Properties & Subtleties**	**Lean Theorems**
`(match` s p t`)` (Workspace query)	One single-read firing per matching occurrence $o \in W$: read o through p, produce t at fresh m	Preserves occurrence multiplicity; stored atoms with live variables require per-use fresh opening	`valid_iff`, `resolved_answer`, `encodedAnswers_eq_reference`, `shared_variable_conflates_queries`
`(collapse` e`)` (Collect all answers)	Store-level observation collecting the bag of resolved answers of all valid single-read firings at C	Cannot be certified by a single 5-port firing; requires a predicate query or store-level observation; sensitive to shared-variable order	`encodedAnswers_antitone`, `OrderEffect.order_matters`, `private_pattern_bag_invariant`, `validity_never_certifies_completeness`, `predicate_query_detects_twin`
`(f` a`)` against `(= (f $x) body)`	Single-read firing reading an opened copy of `(= (f $x) body)` through `(= (f` a`) $r)`, or compiled 1-firing rule schema	Open call `(f $y)` binds caller's $\$y$ via two-way unification; unmatched call falls back via an `absent` check	`callRule`, `Probe.closed_call`, `Probe.open_call_binds_caller`, `Probe.no_equation_returns_call`, `eqStep_iff`
`(add-atom` s a`)` `(remove-atom` s p`)`	Firing at space handle s with produce $= [a]$ (fresh o^+) or take $= [p]$ (consuming $o^- \in W_s$)	Atomic multiset commit; staged sequences commit all-or-nothing in one firing (`defer_one_firing`)	`present_result`, `spaces_fire`, `defer_one_firing`, `no_partial_effects`
`(exec` p `(,` $\vec{s}$`)` `(0 (+` $\vec{a}$`) (-` $\vec{r}$`)))` (MM2 / MORK)	Atomic snapshot-and-commit over a duplicate-free set representation: take `exec` and all matched $\vec{r}$, produce new $\vec{a}$	MORK spaces are *sets* with all-row snapshot semantics; row-by-row firing fails on overlapping removals or creates duplicates	`Result`, `exec_firing_adequacy`, `exec_is_one_firing`, `Probe.row_protocol_refuses_overlap`, `Probe.row_protocol_duplicates`

(`shared_variable_conflates_queries`); stored templates with variables must therefore be opened with fresh variables per use.

2. **Why `collapse` Requires a Predicate Query or Store Observation (`MeTTaOperationsCollapse`).** A five-port rule instance $\langle \text{take}, \text{read}, \text{tell}, \text{checks}, \text{produce} \rangle$ cannot certify that a finite list of read occurrences exhausts *all* matches of an open pattern p: adding a "twin" occurrence containing a fresh constant outside the firing's vocabulary preserves validity of the firing while enlarging the true match set (`twin_preserves_validity`, `validity_never_certifies_completeness`). A birth-cut predicate query (`predicate_query_detects_twin`) or store-level observation (`encodedAnswers`) is therefore essential. Moreover, when a variable $\$x$ is shared between a collapsed query and an outer constraint, operational evaluation order changes the result bag (`OrderEffect.order_matters`), whereas if the pattern's variables are private to the collapsed query, the answer bag is invariant across all consistent stores (`private_pattern_bag_invariant`).
3. **Multi-Space Handles and Read/Write Polarity (`Step.Spaces`, `Step.Permissions`).** When occurrences carry space handles $\text{space} : O \to H$, a single store decomposes into its handle fibers $\text{fiber}(\text{space}, h, W) = \{o \in W \mid \text{space}(o) = h\}$ (`spaces_injective`, `spaces_fire`). In the located first-order step (`Spaces.located`), a firing at handle a that adds no equations (*quiet*) commutes with admission at any distinct handle $b \neq a$ (`Spaces.independent`,

`Controls.quiet_read_independent`), whereas a read at space 1 that binds a shared variable can refuse a read at space 2 sharing that variable (`Controls.shared_variable_crosses_spaces`: spaces separate resources and observations, not shared equations). Under facet rights $\Gamma : H \to \{\texttt{receive}, \texttt{peek}, \texttt{send}, \texttt{observe}\} \to \mathrm{Bool}$ (`Step.Permissions`), a firing lacking the required right is refused (`refused_without_receive`, `refused_without_peek`, `refused_without_send`), a permitted run is noninterfering with spaces where it holds no rights (`permitted_noninterference`), a space without `receive` *only grows* (`fiber_mono_without_receive`, preserving existential witnesses `owned_witness_present`), and a space without `send` *only shrinks* (`fiber_anti_without_send`, preserving universal absence `owned_absence_preserved`).

4 The Certain-Check Proof Calculus

We now present the deductive core of the first-order Transform Calculus: the *certain-check proof calculus* `CertainChecks.Derives` (`Step/ModelDerivation.lean`), together with its underlying region theorems (`FirstOrderChecks.lean`, `PrincipalCheckRegions.lean`) and disequality separation matrix (`Step/Evidence.lean`).

4.1 Syntax, Regions, and Inference Rules of `CertainChecks.Derives`

Fix a first-order signature σ : `LPSignature` with decidable equality on variables, constants, and function symbols, and let $\mathcal{A}_\sigma$ = `FirstOrder.action` σ be the standard substitution action of $\mathrm{Subst}(\sigma)$ on $\mathrm{Term}(\sigma)$. A first-order check c : `Check`$(\mathrm{Term}(\sigma))$ has one of three forms:

$$c \;::=\; \texttt{equal}(t,u) \;\mid\; \texttt{absent}(p) \;\mid\; \texttt{count}(p,n) \qquad (t,u,p \in \mathrm{Term}(\sigma),\ n \in \mathbb{N}).$$

Given an equation store K : `EquationStore`$(\mathrm{Term}(\sigma))$, an occurrence labelling $\ell : O \to \mathrm{Term}(\sigma)$, and a finite world of present occurrences W : `Finset` O, the semantic satisfaction relation $c.\mathrm{Holds}(\mathcal{A}_\sigma, \theta, \ell, W)$ at a substitution $\theta : \mathrm{Subst}(\sigma)$ is:

$$\begin{aligned}
\texttt{equal}(t,u).\mathrm{Holds}(\mathcal{A}_\sigma,\theta,\ell,W) &\iff \theta(t) = \theta(u),\\
\texttt{absent}(p).\mathrm{Holds}(\mathcal{A}_\sigma,\theta,\ell,W) &\iff \forall o \in W,\ \theta(p) \neq \theta(\ell(o)),\\
\texttt{count}(p,n).\mathrm{Holds}(\mathcal{A}_\sigma,\theta,\ell,W) &\iff \left|\{o \in W \mid \theta(p) = \theta(\ell(o))\}\right| = n.
\end{aligned}$$

To decide whether K is consistent *and* entails $c.\mathrm{Holds}(\mathcal{A}_\sigma, \theta, \ell, W)$ across *every* solution $\theta \models K$, `FirstOrderChecks` defines pairwise compatibility and the *certain* and *possible* occurrence regions using the total occurs-checked unifier `unifyTotal` : `EquationStore`$(\mathrm{Term}(\sigma)) \to$ `Option`$(\mathrm{Subst}(\sigma))$:

$$\mathrm{compatible}(K,p,t) := \ (\mathrm{unifyTotal}(K \cup \{p \doteq t\})).\mathrm{isSome}, \tag{2}$$

$$\mathrm{certain}(\mu,\ell,W,p) := \ \{o \in W \mid \mu(p) = \mu(\ell(o))\}, \tag{3}$$

$$\mathrm{possible}(K,\ell,W,p) := \ \{o \in W \mid \mathrm{compatible}(K,p,\ell(o)) = \mathrm{true}\}. \tag{4}$$

Figure 1 displays the inference rules of `CertainChecks.Derives` and `CertainChecks.AdmitsDerivably`. Unlike operational negation-as-failure, Der-Absent does *not* test whether $\mu(p) \neq \mu(\ell(o))$ holds at the principal unifier μ; instead, for every present occurrence $o \in W$, it requires $\mathrm{compatible}(K,p,\ell(o)) = \mathrm{false}$—that is, $\mathrm{unifyTotal}(K \cup \{p \doteq \ell(o)\}) = \mathrm{none}$, refuting the existence of any unifying extension. Similarly, Der-Count requires that the principal unifier μ already equates p with n occurrences in W, and that every remaining occurrence $o \in W$ is incompatible with p under K.

4.2 Worked Example: Absence Under Later Instantiation

Example 4.1 (Guarded Default Firing vs. Open Workspace Occurrence)**.** Consider a MeTTa workspace $W = \{o_1\}$ containing a single open atom $\ell(o_1) =$ `(p $X)` with empty equation store

Judgment Forms (`Step/ModelDerivation.lean`):

- $K;\ell;W \ \vdash_{\text{cc}}\ c$ (`CertainChecks.Derives`): check c is derivably certain from equation store K over labelled world (W,ℓ).
- $C;\ell \ \vdash_{\text{cc}}\ \text{admit}(f)$ (`CertainChecks.AdmitsDerivably`): query-free first-order firing f is derivably admitted at configuration $C = \langle K, W\rangle$.

Inference Rules for Certain Checks (`CertainChecks.Derives`):

$$\frac{\text{unifyTotal}(K) = \text{some}(\mu) \qquad \mu(t) = \mu(u)}{K;\ell;W \ \vdash_{\text{cc}}\ \texttt{equal}(t,u)}\ \textsc{Der-Equal}\ (\texttt{Derives.equal})$$

$$\frac{\text{unifyTotal}(K) = \text{some}(\mu) \qquad \forall o \in W,\ \text{compatible}(K,p,\ell(o)) = \text{false}}{K;\ell;W \ \vdash_{\text{cc}}\ \texttt{absent}(p)}\ \textsc{Der-Absent}\ (\texttt{Derives.absent})$$

$$\frac{\begin{array}{c}\text{unifyTotal}(K) = \text{some}(\mu) \qquad |\text{certain}(\mu,\ell,W,p)| = n \\ \forall o \in W,\ \mu(p) = \mu(\ell(o)) \ \vee\ \text{compatible}(K,p,\ell(o)) = \text{false}\end{array}}{K;\ell;W \ \vdash_{\text{cc}}\ \texttt{count}(p,n)}\ \textsc{Der-Count}\ (\texttt{Derives.count})$$

Derivable Firing Admission (`CertainChecks.AdmitsDerivably`):

$$\frac{\begin{array}{c}\text{Shapes}(C,f) \qquad \text{Enabled}(C.\text{present}, f.\text{selection}) \\ K_f := f.\text{selection.problem}(\ell, f.\text{rule}, C) \qquad \text{unifyTotal}(K_f).\text{isSome} \\ \forall c \in f.\text{rule.checks},\ \ K_f;\ell;C.\text{present} \ \vdash_{\text{cc}}\ c\end{array}}{C;\ell \ \vdash_{\text{cc}}\ \text{admit}(f)}\ \textsc{Der-Admit}$$

Figure 1: The Certain-Check Proof Calculus (`CertainChecks.Derives` and `AdmitsDerivably` in `Step/ModelDerivation.lean`).

$K = []$. Suppose a program defines a specific rule for `(p a)` and a fallback rule f_{def} that emits `(fallback a)` at fresh occurrence o_2 when `(p a)` is absent (`MeTTaOperationsCall.Probe.no_equation_returns_call`, `Step.ProgramUnmatched`):

$$f_{\text{def}}.\text{rule} = \langle \text{take} = [],\ \text{read} = [],\ \text{tell} = [],\ \text{checks} = [\texttt{absent((p a))}],\ \text{produce} = [\texttt{(fallback a)}]\rangle.$$

Let us trace $C;\ell \ \vdash_{\text{cc}}\ \text{admit}(f_{\text{def}})$ through the rules of Figure 1:

1. **Birth-Cut Store:** Since f_{def} takes, reads, and tells nothing, its birth-cut problem is $K_{f_{\text{def}}} = []$. Running unifyTotal([]) returns some(μ) with $\mu = \text{id}$.
2. **Why Principal-Answer Testing Fails:** At $\mu = \text{id}$, we have $\mu(\texttt{(p a)}) = \texttt{(p a)} \neq \texttt{(p \$X)} = \mu(\ell(o_1))$, so certain(id, ℓ, {o_1}, `(p a)`) $= \emptyset$. An engine checking only the principal answer would wrongly fire f_{def} (`open_absence_remains_unaccepted`, `OpenAbsence.the_three_guard_verdicts`).
3. **Refutation Check in** Der-Absent**:** Rule Der-Absent evaluates compatible([], `(p a)`, `(p $X)`) by calling unifyTotal([`(p a)` $\doteq$ `(p $X)`]), which succeeds with some([$\$X \mapsto$ `a`]). Thus compatible = true, possible([], ℓ, {o_1}, `(p a)`) = {o_1} $\neq \emptyset$, and Der-Absent *refuses* f_{def}.
4. **Resolution After Further Information:**
   - If a firing tells $\$X \doteq$ `b` (with `b` $\neq$ `a`), the new store is $K' = [\$X \doteq \texttt{b}]$. Now unifyTotal([$\$X \doteq$ `b`, `(p a)` $\doteq$ `(p $X)`]) = none, so compatible(K', `(p a)`, `(p $X)`) = false, and $K';\ell;\{o_1\} \ \vdash_{\text{cc}}\ \texttt{absent((p a))}$ is derived by Der-Absent (`genuine_incompatibility_establishes_absence_and_zero_count`).
   - Alternatively, in the disequality instance `Disequality.step` (§4.4), if the disequation

frame records $\$X \neq$ `a` while keeping $K = []$, no separating substitution can unify `(p a)` with `(p $X)`, and f_{def} is admitted without binding $\$X$ to any concrete constant (`Evidence.Family.table`).

4.3 Main Theorems of the Proof Logic (with Proof Sketches)

Theorem 4.2 (Absence Refutation Reduction and Exact Count Squeeze)**.** *Let* $\mathcal{X}$: `Frames` Tm Hom *be any frame system,* K *an equation store at frame* a*,* W : `Finset` O *a finite set of present occurrences,* $\ell : O \to \mathrm{Tm}(a)$ *a labelling, and* $p : \mathrm{Tm}(a)$ *a pattern.*

1. ***Absence Needs No Principal Answer*** *(*`DependentFull.entails_absent_iff`*,* `PrincipalCheckRegions.absent_iff_possible_empty`*):* *K entails that p matches no occurrence in W at any solution if and only if for every $o \in W$, the extended store $K \cup \{p \doteq \ell(o)\}$ is unsolvable:*

$$\mathcal{X}.\mathrm{Entails}\big(K,\, \lambda b\, \tau.\, \forall o \in W, \neg\, \mathcal{X}.\mathrm{E}(\mathcal{X}.\mathrm{act}\ \tau\ p,\, \mathcal{X}.\mathrm{act}\ \tau\ \ell(o))\big) \iff \forall o \in W,\ \neg\, \mathcal{X}.\mathrm{Solvable}\big(K \cup \{p \doteq \ell(o)\}\big).$$

2. ***Exact Count Squeeze at a Principal Answer*** *(*`Frames.Principal.entails_count_iff`*,* `PrincipalCheckRegions.count_iff_regions`*):* *Suppose $\mu : \mathrm{Hom}(a, m)$ is a principal answer for K. For any solution $\tau : \mathrm{Hom}(a, b)$ of K, define* $\mathrm{matchedAt}(\tau) := \{o \in W \mid \mathcal{X}.\mathrm{E}(\mathcal{X}.\mathrm{act}\ \tau\ p,\, \mathcal{X}.\mathrm{act}\ \tau\ \ell(o))\}$*. Then:*

$$\mathcal{X}.\mathrm{Entails}\big(K, \lambda b\, \tau.\, |\mathrm{matchedAt}(\tau)| = n\big) \iff |\mathrm{matchedAt}(\mu)| = n \wedge\ \forall o \in W \setminus \mathrm{matchedAt}(\mu),\ \neg\, \mathcal{X}.\mathrm{Solvable}\big(K \cup \{p \doteq \ell(o)\}\big).$$

Proof Sketch. For (1), an occurrence $o \in W$ violates absence at some solution $\tau \models K$ iff τ also solves $p \doteq \ell(o)$, which by definition of `models_append` holds iff $\tau \models K \cup \{p \doteq \ell(o)\}$; negating existential solvability over $o \in W$ yields the universal refutation condition without assuming a principal answer.

For (2), because μ is principal for K, any solution $\tau \models K$ factors as $\tau \equiv \rho \circ \mu$ for some residual morphism ρ (`principal.factors`). By congruence (`preserves`), any equation $\mathrm{act}(\mu, p) \equiv \mathrm{act}(\mu, \ell(o))$ is preserved by ρ, so $\mathrm{matchedAt}(\mu) \subseteq \mathrm{matchedAt}(\tau) \subseteq \mathrm{possibleRegion}(K)$ for every $\tau \models K$ (`Principal.equated`, `principal_region_subset`). ($\Rightarrow$): If $|\mathrm{matchedAt}(\tau)| = n$ for all $\tau \models K$, then choosing $\tau = \mu$ gives $|\mathrm{matchedAt}(\mu)| = n$. If some $o \in W \setminus \mathrm{matchedAt}(\mu)$ had a solution $\tau \models K \cup \{p \doteq \ell(o)\}$, then $\mathrm{matchedAt}(\mu) \subsetneq \mathrm{matchedAt}(\tau)$ would be a strict finite subset (`Finset.ssubset_iff_subset_ne`), forcing $n = |\mathrm{matchedAt}(\mu)| < |\mathrm{matchedAt}(\tau)| = n$ (`Finset.card_lt_card`), a contradiction. ($\Leftarrow$): Conversely, if every $o \notin \mathrm{matchedAt}(\mu)$ has $K \cup \{p \doteq \ell(o)\}$ unsolvable, then no $\tau \models K$ can equate p with any $o \notin \mathrm{matchedAt}(\mu)$, so $\mathrm{matchedAt}(\tau) \subseteq \mathrm{matchedAt}(\mu)$. Combined with $\mathrm{matchedAt}(\mu) \subseteq \mathrm{matchedAt}(\tau)$, the finite sets coincide and $|\mathrm{matchedAt}(\tau)| = |\mathrm{matchedAt}(\mu)| = n$. □

Theorem 4.3 (Soundness, Henkin Completeness, Herbrand Incompleteness, and Derivable Admission)**.** *In the first-order proof calculus* `CertainChecks` *(*`Step/ModelDerivation.lean`*):*

1. ***Executable Equivalence and Semantic Entailment*** *(*`derives_iff_accepts`*,* `derives_iff_entails`*):*

$$(K; \ell; W \vdash_{\mathrm{cc}} c) \iff \mathrm{acceptsCertainCheck}(K, \ell, W, c) = \mathrm{true} \iff \mathcal{A}_\sigma.\mathrm{Consistent}(K) \wedge \mathcal{A}_\sigma.\mathrm{Entails}(K, c.\mathrm{Holds}).$$

2. ***Soundness and Completeness for Free Henkin Models and the Term Algebra*** *(*`derives_sound_free`*,* `derives_complete_generic`*,* `derives_iff_termModel`*):* *If $K; \ell; W \vdash_{\mathrm{cc}} c$, then* $\mathrm{checkGuard}(\ell, W, c)$ *holds at every point satisfying K of every free Henkin model in every universe. Conversely, if K has a generic point and* $\mathrm{checkGuard}(\ell, W, c)$ *holds at every*

generic point satisfying K, then $K;\ell;W \vdash_{\mathrm{cc}} c$. In particular, one set model—the free term algebra $\mathrm{termModel}(\sigma)$ *whose points are substitutions $\theta : \sigma.\mathrm{vars} \to \mathrm{Term}(\sigma)$—is both sound and complete (*`derives_iff_termModel`*), and for distinct constants $c \neq d$, $\neg(K;\ell;W \vdash_{\mathrm{cc}} \mathtt{equal}(c,d))$ (*`not_derives_clash`*).*

3. ***Incompleteness of the Ground Herbrand Model (***`oneConstant_separates`***):*** *The ground Herbrand model* $\mathrm{herbrandModel}(\sigma)$ *is sound (*`derives_herbrand_sound`*) but* not *complete: over the signature* oneSig *with one constant* `c`*, no function symbols, and variables* $\mathbb{N}$*, the check* $\mathtt{equal}(X_0, \mathtt{c})$ *holds at every ground assignment $g : \mathbb{N} \to \mathrm{GroundTerm}(\mathrm{oneSig})$ over the empty store $K = []$, yet $[];\ell;\emptyset \not\vdash_{\mathrm{cc}} \mathtt{equal}(X_0, \mathtt{c})$.*
4. ***Admission as Formal Derivation (***`admitsDerivably_iff`***):*** *For every query-free first-order firing f ($f.\mathrm{queries} = []$),*

$$(C;\ell \vdash_{\mathrm{cc}} \mathrm{admit}(f)) \iff f.\mathrm{Valid}\ \mathcal{A}_\sigma\ \ell\ C.$$

Proof Sketch. Part (1) combines the soundness and principality of total occurs-checked unification (`FirstOrder.accepted_principal`, `FirstOrder.refused_iff_inconsistent`) with Theorem 4.2 (`equal_iff_principal_answer`, `absent_iff_possible_empty`, `count_iff_regions`). For Part (2), any free Henkin model M realizes every syntactic substitution action homomorphically (`Frames.HenkinModel.sound`), yielding soundness across all universes; completeness follows because the generic model Frames.generic (and Concrete $\mathrm{termModel}(\sigma)$) contains the identity substitution Subst.id and all substitution instances as points (`Frames.complete`, `termModel_realizes`). For Part (3), in oneSig every closed ground term $g(0) : \mathrm{GroundTerm}(\mathrm{oneSig})$ is necessarily GroundTerm.const () by case analysis (since functionSymbols = Empty), so $g \models (X_0 = \mathtt{c})$ for all ground g; however, in termModel(oneSig), the identity substitution $\theta_{\mathrm{id}}(X_0) = \mathrm{Term.var}\ 0 \neq \mathrm{Term.const}\ ()$ models $K = []$ and falsifies $X_0 = \mathtt{c}$, refuting derivability. Part (4) unfolds `firstOrder_admits_iff`: when $f.\mathrm{queries} = []$, universal satisfaction of $f.\mathrm{rule.checks}$ over a solvable K_f is equivalent by Part (1) to $\mathrm{unifyTotal}(K_f).\mathrm{isSome}$ plus $K_f;\ell;C.\mathrm{present} \vdash_{\mathrm{cc}} c$ for each $c \in f.\mathrm{rule.checks}$. □

4.4 Conflicting Evidence vs. Inconsistent Stores and the 4×4 Separation Matrix

A critical distinction in non-monotonic and epistemic reasoning is that *conflicting evidence in the workspace is data, whereas an inconsistent equation store is unsolvable knowledge* (`Step/Evidence.lean`). Adding both an evidence atom `ev(p)` and an opposing atom `opp(p)` to the workspace leaves the equation store consistent (`Evidence.Rows.conflict_consistent`): a firing `weigh` that reads both is admitted (`weigh_admitted`), whereas a defeasible firing `accept` that reads `ev(p)` under an absence guard `absent(opp(p))` is admitted when `ev(p)` is alone (`accept_admitted_with_evidence`) and refused once `opp(p)` arrives (`accept_refused_with_opposition`). By contrast, telling both $X \doteq \mathtt{true}$ and $X \doteq \mathtt{false}$ into the equation store is refused (`tell_false_refused_after_true`), and a state containing both admits no firing at all (`both_tells_admit_nothing`).

Furthermore, over the disequality step `Disequality.step` (`Step/Disequality.lean`), `Evidence.Family` proves that four distinct epistemic states regarding "not finding `p(a)`" have pairwise distinct behavioral profiles across four atomic firing probes (`Evidence.Family.table`, `Evidence.Family.profiles_distinct`, Table 3). In particular, Member 3 (`Member.incomplete`, where `p(X)` is present with X open) refuses the absence guard "no `p(a)`" while admitting the tell $X \doteq \mathtt{a}$ and the unifying read `p(a)`; a partial scan that skips the open row $o = 2$ would falsely license the absence guard (`incomplete_scan_would_license`, `skipping_scan_not_covering`). Member 4 (`Member.disequal`, where `p(X)` is present under the disequation frame $X \neq \mathtt{a}$) derivably admits the absence guard "no `p(a)`" while refusing both the tell $X \doteq \mathtt{a}$ (`same_refused`) and the read `p(a)`. In general, as proved in `Disequality.presents_iff`, a satisfiable disequation store that some substitution violates (such as $X \neq \mathtt{a}$) has no single presenting substitution; elevating disequation stores into the frame category `Disequality.frames` recovers principal answers as morphisms into the residual disequation frame (`Disequality.principal`, `solvable_iff_separates`).

Table 3: The 4×4 Evidence Separation Matrix proved in `Evidence.Family.table` and `Evidence.Family.profiles_distinct` (`Step/Evidence.lean`) over the disequality step (`Disequality.step`) with rows $\ell(1) =$ `opp(a)` and $\ell(2) =$ `p(X)`.

Member (`Evidence.Family.Member`)	**Workspace & Frame State**	**Guard** **no** `p(a)`	**Read** `opp(a)`	**Tell** $X \doteq$ `a`	**Read** `p(a)`
1. `Member.covered`	Nothing present ($W = \emptyset, D =$ [])	Admitted	Refused	Admitted	Refused
2. `Member.refuted`	`opp(a)` present ($W = \{1\}, D =$ [])	Admitted	Admitted	Admitted	Refused
3. `Member.incomplete`	`p(X)` present, X open ($W = \{2\}, D =$ [])	Refused	Refused	Admitted	Admitted
4. `Member.disequal`	`p(X)` present ($W = \{2\}, D =$ $[X \neq$ `a`])	Admitted	Refused	Refused	Refused

5 Proof Calculi and Fair Search as Firings

Because `Interface.Step` separates non-consuming reads from consuming takes and supports solution-level guards, classic automated theorem proving calculi and search engines embed directly as firings and runs of the step.

5.1 Resolution, Guarded Superposition, and Ground Completion

1. **First-Order Resolution (`Step.ResolutionFiring`, `Step.ResolutionReadPort`, `Step.ResolutionAnswers`):** Background clauses are kept in place and read via the read port (`ReadPort.resolve_valid_iff`), while active goal clauses are taken and replaced by their resolvents (`resolve_sld_step`). Switching literal selection order yields trace-equivalent firings (`resolve_switch_traceEquiv`), and N-step resolution derivations are sound and refutationally complete (`NDerivation.complete`). By contrast, attempting to split a positive disjunction $P \vee Q$ in a single deterministic firing without branching leaves an unconsumed residue (`Controls.split_leaves_residue`).
2. **Superposition and the Upward Closure of Co-Stable Guards (`Step.SuperpositionGuarded`, `Step.SuperpositionCompleteness`, `Step.SuperpositionGenerator`):** Superposition requires ordering side conditions such as $\neg(r \succeq l)$ that lie outside the syntax of `Check`. `Superposition.guarded` extends `AnnotatedRuns.firstOrder` with solution predicates $G : S \to \mathrm{Prop}$. A subtlety arises at the birth cut: while positive equations are *stable* under instantiation ($\mathrm{Stable}(G) := \forall \nu\, \sigma,\ G(\sigma) \implies G(\nu \circ \sigma)$), negated ordering constraints $\neg(r \succeq l)$ are *co-stable* ($\mathrm{CoStable}(G) := \forall \nu\, \sigma,\ G(\nu \circ \sigma) \implies G(\sigma)$). Asking a co-stable guard at *every* solution of K is strictly stronger than asking it at the principal unifier μ (`at_principal_of_entails`). `Step.SuperpositionGuarded` resolves this by taking the *upward closure* of G:

$$\mathrm{Closure}(K, G)(\sigma) \ := \ \exists \tau \models K,\ \ \mathrm{Factors}(\sigma, \tau) \ \wedge \ G(\tau).$$

   $\mathrm{Closure}(K, G)$ is always stable (`Closure.stable`), and at any store K with a principal unifier μ, universal entailment of $\mathrm{Closure}(K, G)$ over all solutions is equivalent to $G(\mu)$ at the MGU (`entails_closure_iff`)—and over a minimal complete set of unifiers U, to $\forall \mu \in U,\ G(\mu)$ (`entails_closure_iff_complete`). On ground clauses, the superposition calculus is refutationally complete via Bachmair–Ganzinger model construction (`SuperpositionCompleteness`), simplification takes redundant clauses (`simplify_takes_redundant`), and the fair given-clause loop is complete (`givenClause_complete`).
3. **Ground Knuth–Bendix Completion (`Step.CompletionFiring`, `Step.CompletionCorrectness`, `Step.CompletionCommutativity`):** All six rules of ground Knuth–Bendix completion [20] over

GroundTerm(σ)—Deduce, Orient, Delete, Simplify, Compose, and Collapse—are formalized as firings (`Move.transition`), where inference rules read equations and simplification rules take and replace them. Every fair derivation from a decreasing initial state converges to a canonical convergent system deciding the equational theory (`Derivation.convergent`, `Derivation.decides`), whereas commutativity can never be oriented by any symmetric reduction order (`no_symmetric_order_orients`, `commutativity_fails`).

5.2 The Search Outcome Algebra, Guard Polarities, and Fair Exploration (`Step.Search*`, `Step.GuardFragments*`, `Step.FairSearch*`)

When multiple firings contend at a configuration, bounded exploration and behavioral guards are governed by `SearchOutcome.Outcome` (`Step/SearchOutcome.lean`, `Step/SearchSpace.lean`, `Step/SearchPolicy.lean`) and the polarity-graded guard suite (`Step/GuardFragments.lean`, `Step/GuardFragmentsOpen.lean`, `Step/GuardLowering.lean`, `Step/MeteredGuards.lean`, `Step/FairSearch.lean`, `Step/FairSearchIdentity.lean`, `Step/FairSearchGuards.lean`):

- **Commutative Nondeterminism Monad & Three-Valued Verdicts:** Search outcomes carry a bag of answers and a termination status (`complete` vs. `cutOff`), forming a commutative monad up to multiset equivalence (`Outcome.choice_comm`, `Outcome.bind_assoc`). The three-valued verdict (`Outcome.verdict`) strictly separates resource exhaustion (`.cutOff`) from finite failure (`.refuted`): a cut-off search never yields a refutation verdict (`Outcome.verdict_ne_refuted_of_cutOff`), proved refutations are sound across all budgets (`refuted_sound`), and conflating cutoff with refutation is unsound (`Controls.misread_refutes`). While depth-first search can be trapped forever in a left-recursive branch (`Controls.dfs_loses`), breadth-first search and iterative deepening find every reachable answer at a finite budget (`bfs_complete`, `deepen_complete`; tabling controls in `Step.SearchTabling` and `Step.ResolutionTabling`).
- **Guard Polarities, Park Certificates, and Exact Lowering (`Step.GuardFragments`, `Step.GuardLowering`):** Behavioral guards over a datum's transition graph split into reachability (Σ, least fixed point, witnessed by one finite path `eventually_iff_exists_checkPath`) and safety/invariance (Π, greatest fixed point, refuted by one finite path `not_always_iff_exists_checkPath`). Certifying a Π guard requires a finite post-fixed point (Park induction), which exists if and only if the datum holds and has finitely many reachable states (`exists_invariant_iff`, `always_iff_exists_checkInvariant`); on cyclic finite-state data, the three-valued closure verdict `verdictAlways` commits where depth-bounded tree search is cut off at every budget (`always_iff_exists_commit`, `tree_search_never_certifies_loop`), whereas infinite-state data remain suspended at every finite budget and require an offline symbolic invariant (`suspended_of_infinite`, `always_iff_of_symbolicInvariant`). On open data, Σ guards are stable and decided at a principal answer (`eventually_entails_iff`), whereas Π guards are not (`always_not_stable`). Replacing a Π specification by a checked certificate preserves and reflects all runs modulo certificate erasure whenever configurations preserve the finite-state invariant (`runs_spec_iff_lowered`, `certified_runs`), whereas unfolding over a subset not closed under successors admits unsoundly (`closure_free_unsound`, `closure_free_admits_wrongly`). Guard cost must be metered as a site charge on the certificate (`certified_metered_iff`) rather than by store-scan size, which breaks interleaving invariance (`scan_price_order`).
- **Fair Search Makes the Certificate & Branching-Time Proof Identity (`Step.FairSearch*`, `Step.FairSearchIdentity`):** Monotone search firings that only read and make saturate the reachable state space under weak fairness (`fires_or_preempted`, `fair_search_certifies`, `answer_sound`, `answer_eventually`). By contrast, search firings that *take* a goal (competing OR-receives) can fail to prove even along a strongly fair run when a proof is reachable (`race_stronglyFair`, `race_never_proves`, `race_proof_reachable`), whereas adding a repair firing yields the exact same reachable stores and converges under weak fairness (`reachability_does_not_decide_fair_completeness`). Fair completeness requires $\mathsf{AG}\,\mathsf{EF}$ (persistent reachability), which is preserved by observed bisimilarity with zero axioms

(alwaysPossibly_of_bisimilar, search_identity_is_branching).

6 Programs on the Step: Compilation, Interpretation, and Binders

The Step.Program* suite establishes how functional and rewrite programs execute over the Transform Calculus on the ordinary first-order step and the higher-order contextual step:

1. **Substitution Evaluation vs. Environment Machine (Step.ProgramSubstitution, Step.ProgramSubstitutionMachine):** Following Plotkin [25], evaluation of closed terms (Term) by environment frames and explicit closures corresponds forward and backward to substitution-based evaluation (plotkin_forward, plotkin_backward, plotkin_closed) and to the multi-step abstract machine on the step (machine_iff).
2. **Compiled Rule Schemas vs. Stored-Equation Interpreter (Step.ProgramEquations, Step.ProgramCompiledInterpreter, Step.ProgramCompiledObstruction):** A program of pattern equations (= lhs rhs) executes in two ways:
   - *Compiled execution* (ProgramEquations.compile): each program equation compiles to its own rule schema, rewriting a call in a single firing (eqStep_iff, equations_execution; versioned updates in vstep_iff, vexecution). On open calls, two-way unification computes answers in one firing uniquely up to MGU variable renaming [11] (mgu_unique_up_to_renaming). When no equation matches, a default branch fires under an absent guard once the unmatched call is sufficiently instantiated for absence to be stable (absence_stable, unmatched_fire in Step.ProgramUnmatched).
   - *Stored-equation interpretation* (ProgramCompiled.metaProg, Step.ProgramInterpreterConverseJoin): by no_fixed_one_firing_interpreter (Step/ProgramCompiledObstruction.lean), no fixed finite family of rule schemas with bounded pattern variables can interpret arbitrary stored equations in a single firing. Instead, metaProg is a fixed 16-equation interpreter that reads stored equation codes (eqCode) and executes administrative matching (mt, ck) and building (bj, rd) firings. On ground calls of a range-restricted program, *one compiled firing corresponds to a complete multi-firing interpreter run* (compiled_iff_interpreted), an interpreter step embeds into a firing on the step (interpreter_join_on_step), and on open calls with data rows (DataRows), interpreted and compiled answers agree *both ways* up to MGU variable renaming (interpreter_join_converse, interpreter_join_forward, interpreter_join_iff).
3. **Nested Calls, Call-by-Need Sharing, Inert Quotation, and Binders (Step.ProgramNested*, Step.ProgramNeed*, Step.ProgramReflection, Step.ProgramBinders*, Step.ProgramHosts):** Nested expression calls reduce via an 8-rule stack operational semantics KStep and tree semantics CStep (Step/ProgramNestedSource.lean), which are sound and complete for nested execution on the step (nested_execution, reference_complete), while Launchbury call-by-need evaluation with heap-shared thunks corresponds both ways to step runs (nstep_fires, fires_nstep, need_on_step, need_eval_on_step in Step.ProgramNeedStepAdequacy). Sequences of workspace effects (add-atom followed by remove-atom) stage into a single atomic firing with no partial intermediate effects (defer_one_firing, no_partial_effects). Quoted expressions (quote e) are completely inert (quotation_inert), and activating a quoted expression runs the multi-step evaluation machine on the step (activate_quote_on_step), while raw syntactic variable-reflection probes violate semantic congruence (reflect_not_semantic, ordinary_by_rows). On the higher-order contextual guarded step (ContextualGuardedStep), pattern equations rewrite under λ-binders via quoted contextual terms (quoted_rewrites_iff, quoted_execution_iff in Step.ProgramBindersQuoted) and obey a general raising grammar law (raise_answers_iff, root_runs_on_step in Step.ProgramBindersRaisingConverse, Step.ProgramBindersRaisingConverseGrammar): absence of a matching equation at a contextual firing is decided by certified refutation on the Miller pattern fragment (guarded_

admitted_iff, certified_guarded, certifies_guarded_of_absent, absence_stable), unmatched calls under binders return themselves (returns_iff, returned_is_final, evaluates_iff in Step.ProgramBindersUnmatched) and agree with the first-order host on embedded rows (absence_agrees, return_admitted_iff in Step.ProgramHosts), and both general innermost reduction and deterministic leftmost-innermost reduction under binders are sound, complete, and decided (innermost_step_iff, innermost_execution_iff, innermost_of_certified, innermost_redex_certified, leftmost_step_iff, leftmost_execution_iff in Step.ProgramBindersStrategy, Step.ProgramBindersStrategyDecision, Step.ProgramBindersLeftmostStep).

7 Concurrency, Races, and Behavioral Equivalence

Theorem 7.1 (Concurrency Diamonds Across Step Instances)**.** *Across the first-order, contextual, and dependent instances of* Interface.Step*:*

1. ***First-Order and Contextual Diamonds (***AnnotatedRuns**,** Step.ContextualDiamond***):*** *Two firings with disjoint consumed footprints, mutually compatible birth-cut stores, and non-interfering query regions commute up to state isomorphism (*swap_valid*,* traceEquiv_executable*,* Step.ContextualDiamond*).*
2. ***Dependent Plain, Helper, and Telescope Diamonds (***Step.DependentFullIndependence**,** Step.DependentFullSwap**,** Step.DependentFullTelescopes**,** Step.DependentFullRaceHelpers***):*** *In* DependentFull.step*, two plain firings at frame Δ commute iff their mutual residuals are admitted (*local_diamond*,* independent_iff_residuals*). Two helper firings introducing fresh metavariables $x : A$ and $y : B$ commute via the context swap isomorphism $\Gamma, x : A, y : B \cong \Gamma, y : B, x : A$ (*helper_diamond*), whereas if B depends on x, no independent residual exists before x is introduced (*no_residual_of_mentions*). Multi-variable helper telescopes $\vec{H}_1, \vec{H}_2$ commute similarly (*telescope_diamond*), and isomorphic helper states induce branching bisimilarity (*helper_branchingBisimilar_of_same*).*

Milner's Race Pair and the Behavioral Spectrum (Step.RacePair**,** Step.RaceSpectrum**).** When two firings compete for a shared occurrence, trace equivalence and answer-bag equality are strictly coarser than bisimilarity. In Step/RacePair.lean, Milner's classic pair [24] $P = a.(b + c)$ and $Q = a.b + a.c$ is realized directly as firings on the step: P and Q are trace-equivalent (traceEquivalent_P_Q) and produce identical answer bags at every depth (collect_P_eq_collect_Q), yet for $b \neq c$ they are not bisimilar (not_bisimilar_P_Q). Across Step/RaceSpectrum.lean and Step/RaceHistories.lean, the formalization establishes the strict hierarchy:

$$\text{History-Preserving Bisim.} \subsetneq \text{Step Bisim.} \subsetneq \text{Interleaving Bisim.} \subsetneq \text{Trace / Bag Equivalence.}$$

Graded Choice, Causal-History Factorization, and Interventions (Step.CausalHistories**,** Step.Contention**,** Step.Intervention**).** When enabled firings carry positive rates, total exit rates add across parallel components (total_par), and summing the global jump-chain likelihood over all interleavings (shuffles) of two non-interfering maximal runs α, β yields the exact product of their component history probabilities (sum_shuffles, history_factorizes, history_factorizes_three, step_history_product, local_exact). Crucially, on a step with absence guards, contention must be defined as step interference rather than consumed-occurrence sharing alone (not_independent_of_contends): when one firing emits an atom that another firing's absence guard forbids, sharing-only normalization assigns total mass 2 whereas interference-aware normalization recovers total mass 1 (inhibition_control). On inhibition-free firings, no external context can force a positive-weight race to certainty (no_context_forces), whereas with an absence guard a single added context atom performs a hard causal intervention (forcing_control, context_performs_do). Scoped interventions $\mathsf{do}(\varphi)$ compose commutatively (restrictWeight_comm), lower exactly on static

alphabets (`prob_restrictWeight`), and when placed at a located space i reach only firings holding the `send` permission at i (`doAt_of_no_send`).

8 Contextual & Dependent Instances and Formalization Status

We now summarize the higher-order contextual and dependent instances of `Interface.Step`, the open versus closed admission distinction, the three fibre-join theorems, and the exact boundary between completed proofs and active work in progress.

8.1 Contextual and Dependent Instances and the Open/Closed Admission Trap

1. **Contextual Second-Order Frames (`Interface.Frames.contextual`):** In `Step/Interface.lean`, `Frames.contextual` takes declaration frames $F = (\text{Names}, \text{arity})$ as objects and contextual assignments as morphisms acting on observations by capture-avoiding instantiation. Because the frame `firstOrderFrame` S (one parameterless metavariable per sort) is weakly terminal (`firstOrderFrame_weaklyTerminal`), semantic solvability of any contextual equation store is witnessed at a first-order frame without needing higher-order helpers (`contextual_solvable_iff_firstOrder`), and on an inhabited frame coincides with endomorphic consistency (`contextual_solvable_iff_consistent`).
2. **Typed Dependent Frames and Subject Reduction (`Step.TypingStates`, `Step.DependentFull`):** On the dependent helper step (`Dependent.helperStep`), state typing is preserved along every run (`TypingStates.run_preserves`), and along runs from a simple frame with empty knowledge, the non-dependent simple fibre decides admission identically to the dependent helper step (`Dependent.simpleStep_admits_iff`, `Dependent.admits_iff_along`). On the full dependent step (`DependentFull.step`), every admitted firing preserves typing of all surviving and newly made rows at the extended telescope $\Delta = f.\text{helpers.frame}$ (`Firing.subject_reduction`).
3. **Open vs. Closed Admission (`Step.DependentClosed`):** Given a designated closed frame point (the empty context $\cdot$), `Step.ClosedAdmits` *replaces* symbolic solvability and symbolic entailment in Eq. (1) with $\mathcal{X}.\text{ClosedSolvable}(\text{point}, \text{information}(f))$ and $\mathcal{X}.\text{ClosedEntails}(\text{point}, \text{information}(f), \text{Holds}(f, \text{point}))$:

$$\begin{aligned} P.\text{ClosedAdmits}(\text{point}, f) \iff\ & P.\text{Shapes}(f) \ \wedge\ \text{Enabled}(P.\text{present}(s), P.\text{selection}(f)) \\ & \wedge\ \mathcal{X}.\text{ClosedSolvable}(\text{point}, K_f) \\ & \wedge\ \mathcal{X}.\text{ClosedEntails}(\text{point}, K_f, \text{Holds}). \end{aligned}$$

   While symbolic solvability ignores the type A of a fresh metavariable (`solvable_extend_iff`: K lifted to $\Gamma, x : A$ is symbolically solvable iff K is solvable at Γ), closed solvability requires A to be inhabited at a closed solution of K (`closedSolvable_extend_iff`). Consequently, at a frame $x : A$ where A has no closed term, the empty store is symbolically solvable but not closed-solvable (`obligation_trap`), and a guard-free helper firing is symbolically admitted yet closed-refused (`trap_helper`). When K has a principal answer into a closed-inhabited frame, symbolic and closed solvability coincide (`closedSolvable_iff_of_principal`).

8.2 The η-Long Fibre and the Three Fibre-Join Theorems

When embedding a second-order step into a dependent type theory with Π, Σ, and Id types, two obstacles arise if $\beta\eta$-disciplines are misconfigured: (i) if object-level β-redexes are conflated with meta-level instantiation, solvable equations become unsolvable (`ObjectTheory.object_beta_unsolvable`); and (ii) without η-conversion (`ContextualDependentEta.function_eta`, `pair_eta`), even simple pattern equations such as $(\lambda y.\, M(y)) \doteq N$ have two incomparable minimal solutions ($N \mapsto \lambda y.\, M(y)$ and $M \mapsto \lambda y.\, N(y)$) and therefore *no principal unifier* (`pattern_no_principal`).

In `ContextualDependentFibre.lean`, the composite frame map `fibre` embeds second-order list frames into the dependent unifier's typed $\beta\eta$ frames (`ContextualDependentEta.frames`) as the *η-long fibre*: under a closed realization, translated models coincide on translated assignments (`fibre_models_iff`), universal asks reflect (`fibre_entails_reflect`), and certified dependent refutations transfer back to second-order unsolvability (`fibre_refutation`). Rather than a single merged statement, the transfer of firing admission across `fibre` is proved in *three distinct theorems* with explicit hypotheses:

Theorem 8.1 (The Three Fibre-Join Theorems)**.** *Let Φ = `fibre` be the η-long fibre map from the second-order step at list frame M into* `DependentFull.step` *under a closed, sort- and operator-injective realization:*

1. ***Equation-Guard Join Where the Full Pattern+Prune+Exchange Solver Decides*** *(`DependentFullJoinFull.fibre_admission_transfer_of_full`)**:** Because translated metavariable types are closed Π-chains of sort constants (`constChain`), both strengthening (`invert_chain`) and binder pruning (`dropPis_chain`) preserve translated telescopes (`full_keeps_translated`). Consequently, for any firing whose guards are equation guards and whose translated information store is decided by* `ContextualDependentPatternSolver.fullSolver` *(`decidedInFibre_of_full`), second-order admission is preserved and reflected by* `DependentFull.step`*. (Normalized and permuted solver variants are covered similarly by* `normalizing_keeps_translated` *and* `permuting_keeps_translated` *in* `Step.DependentFullJoinSolvers`*.)*
2. ***Absence-Check Join Under*** `EndoDecided` *(`DependentFullFibre.admits_translate_iff`, `result_translate`)**:** For any first-order firing f of the second-order action without census queries (f.queries = []) and without counting checks, whenever the dependent unifier decides the translated birth-cut information* and *each translated absence extension $K_f \cup \{p \doteq \ell(o)\}$ by an endomorphic principal answer or a refutation (`EndoDecided`), first-order admission is equivalent to dependent admission of* translateFiring(f) *(together with the first-order fixed-labelling shape on produced occurrences), and the post-states correspond (`result_translate`).*
3. ***Exact Count-Check Join Under*** `EndoDecided` *(`DependentFullCount.admits_translate_iff_count`)**:** Using Theorem 4.2(2) (`Frames.Principal.entails_count_iff`), the exclusion of counting checks in (2) is removed: for any firing f with f.queries = [] (allowing equality, absence, and exact count checks), under* `EndoDecided` *on the translated problem and on each absence and count extension, first-order admission is preserved and reflected by* `DependentFull.step`*.*

8.3 Honest Status of Dependent Unification

In accordance with strict formalization hygiene, we explicitly separate what is machine-checked in Lean 4 today from what remains open in the dependent unifier (`ContextualDependent*`):

- **Completed and Sorry-Free:** All 270+ modules cited in this paper compile in Lean 4 with **zero `sorry`** and only standard Lean axioms (`propext`, `Quot.sound`, `Classical.choice`). For the dependent unifier specifically, the repository proves the well-founded weight/rank measure (`ContextualDependentRank.weightRank_wf`), conversion-aware spine recognition and eventual recognition when declared argument types agree (`ContextualDependentRecognitionConversion.recognizeConv`, `patternShape_some`, `recognizeConv_eventually` in `ContextualDependentPatternFiringRecognition`, alongside the cumulativity control `cumulative_unread`), in-place rule application (`ContextualDependentInPlace.inPlaceRule`), structural decomposition under $\Pi/\Sigma/\mathrm{Id}$ formers, dependent spines, and local-head projection spines (`ContextualDependentSpinesManyFormers.underParts_exact`, `ContextualDependentSpinesManyDependent.dependent_exact`, `proj_exact`), certified η for Σ on rigid neutrals (`expandOne_solves`), skeleton and kind clashes (`headClash_not_equal`), occurs-check refutation across formers, identity endpoints, and universe-domain spine arguments (`ContextualDependentOccursFull.occurrence_not_typeEq`, `ContextualDependentOccursFullUnder.prefixed_not_equal`, `inside_not_typeEq`, `occursAt_`

Table 4: Comparison of the Transform Calculus with related rewriting, process, and proof systems.

System / Calculus	State Substrate	Read vs. Consume	Negation / Absence / Count	Unification & Binding
Transform Calculus (This work)	Multiset of O with handles (`Spaces`)	Explicit take vs. read ports	Birth-cut universal entailment $\forall\sigma \models K$ (`CertainChecks.Derives`)	Modular `Frames`: FO, disequality, contextual HO, dependent $\beta\eta$
ρ-Calculus / Rholang [21]	Parallel process multiset	Consume in core ρ; Rholang adds `peek` (`<-`)	No native absence or cardinality guard in core ρ	Syntactic quote/drop $\equiv_N$; no open unification store K
MeTTa (HE / PeTTa / CeTTa) [15]	Atomspace (multiset or indexed trie)	Read via `match`; consume via `remove-atom`	Operational via `collapse`; order-dependent on shared vars	Two-way FO unification; no native birth-cut universal check
MM2 / MORK [15]	**Set** of atoms (`Finset Atom`, with vars)	Atomic snapshot-all-rows $+/-$ commit	Set subtraction on snapshot; no multiset occurrence IDs	Unification across sources against snapshot (`MeTTaOperationsExec`)
CHR & CHR$^\neg$ [13, 26]	Constraint multiset + global store	Simpagation ($H_1 \setminus H_2 \iff G \mid B$)	Operational ω_r negation-as-absence in CHR$^\neg$	Built-in FO/CLP constraint theory $\mathcal{CT}$
Maude / Rewriting Logic [22]	Algebraic term modulo Ax (ACU)	Consuming rewrite; read by re-emitting	Reflective `metaSearch` at meta-level	Order-sorted matching modulo Ax (and narrowing)
CHAM / Gamma & MSR [3, 5, 8]	Chemical multiset / typed multiset	Reaction consumes; membrane/catalytic read	Reaction condition local to selected molecules	Pattern matching (CHAM) or dependent LF types (MSR)
CLF & Lolli / LO [9, 18, 2]	Linear Δ + unrestricted Γ	Linear ($\multimap$) vs. persistent (!)	Monotone linear proof search; no $\forall\sigma$ absence/count guard	$\exists x.\, A$ in CLF monad; Miller L_λ HO patterns
λProlog, CLP(X) & FV Tableaux [23, 19, 12]	Goal stack / tableau branch + store K	Clause backchaining / branch expansion	Universal branch closure / rigid E-unification [14]	CLP(X) solver or L_λ HO pattern unification

`not_typeEq`), the equation census and stratified closure class (`census_fragment`, `Stratified`, `stratified_of_fragment`, `Stratified.splice`), and outcome conservativity of the extended rule chain over earlier solver families (`drive_conservative`, `arguments_decided_of_any`).

- **Open Work in Progress (Dynamic-Pattern Decisiveness `E1`):** While the first-order unifier `unifyTotal` is unconditionally total and complete (`FirstOrder.accepted_iff_consistent`), *full algorithmic decisiveness of the dependent unifier on the dynamic higher-order pattern fragment [1] is open.* In `ContextualDependentFragmentWorklist` and `ContextualDependentAcceptance`, `Settled.ofConst` and `argumentsSettledOfConst` remain conditional on an eventual-constancy / acceptance obligation over `Stratified`: while the rigid census rows are read (`headClashAt_complete`), three rigid shapes remain unread (`rows_unread`: flexible-base spines, nested projections, and applied projections) and progress of the flexible pattern rules at sufficient budget together with closure of `Stratified` under answers is active ongoing work.

9 Comparison with Related Calculi and Systems

Table 4 summarizes how the Transform Calculus compares with nine major families of rewriting, concurrency, and logic programming systems.

1. Distinction from the ρ-Calculus and Rholang. In the ρ-calculus [21], names are quoted processes ($\ulcorner P \urcorner$), communication is linear consumption, and there is no persistent equation store K or birth-cut universal guard. (Rholang extends core ρ with persistent peek `for (y «- x)` and spatial-behavioral where-clauses.) In `ContextualCommunicationAdequacy.lean` and `ContextualCommunicationReflection.lean`, the core ρ-calculus embeds into the Transform Calculus with exact step and path adequacy: given a normalized occurrence representation whose background equations hold at the identity model, a source ρ-step exists if and only if an interpreted firing exists (`source_step_iff_firing`, `source_path_iff_firings`). Moreover, in complete agreement with `Mettapedia/papers/rho.tex`, the formalization separates name-level quote-drop ($\ulcorner *x \urcorner = x$, `name_quote_drop_is_equation`) from operational process-level drop-quote ($*\ulcorner P \urcorner \to P$): in the core communication profile without the reflection rule, $*\ulcorner \mathbf{0} \urcorner$ has no communication step and no core occurrence firing (`dropped_null_has_no_core_occurrence_execution`), whereas in the extended profile `withReflection` it fires as an explicit operational step that is not a syntactic conversion (`process_drop_source_step`, `operational_drop_need_not_be_conversion`).

2. Distinction from MeTTa (HE, PeTTa, CeTTa) and MM2/MORK. Existing MeTTa runtimes evaluate queries operationally along a search schedule, which makes `(collapse ...)` sensitive to whether a shared variable is constrained before or after the collapse (`OrderEffect.order_matters`). The Transform Calculus gives a schedule-independent declarative admission criterion (`admits_iff`) and proves when operational shortcuts are sound (e.g., private-pattern invariance `private_pattern_bag_invariant` and covered scans `absent_accepts_iff_of_covered`). Compared to MM2/MORK (`MeTTaOperationsExec`), MORK stores expressions (including expressions with variables) in a *set* (`Finset Atom`) and executes `exec` by matching all source rows against a snapshot, writing all + instances, and subtracting all − instances (`Result`, `exec_firing_adequacy`, `exec_is_one_firing`). Executing MORK's rules one row at a time in a multiset either fails when two rows remove the same atom (`Probe.row_protocol_refuses_overlap`) or creates duplicate atoms that a set collapses (`Probe.row_protocol_duplicates`).

3. Distinction from CHR, CHR$^\neg$, CHAM/Gamma, MSR, CLF, and Free-Variable Tableaux. CHR's simpagation rules [13] inspired our separation of read (kept) and take (removed) ports, while CHR$^\neg$ [26] adds operational negation-as-absence under the refined semantics ω_r. Our `CertainChecks.Derives` calculus makes absence and exact counting *model-theoretically sound and complete* with respect to free Henkin models (`derives_iff_termModel`) and characterizes firing admission as formal derivability (`admitsDerivably_iff`). Similarly, whereas CHAM [5], Gamma [3], MSR [8], Lolli/LO [18, 2], and CLF [9] (which supports existential quantification $\exists x.\, A$ inside its concurrency monad) are monotone with respect to workspace extension, the Transform Calculus supports non-monotonic birth-cut absence and exact cardinality guards while maintaining explicit occurrence identities and formalizing the universal-closure problem familiar from free-variable tableaux [12, 16] and rigid E-unification [14].

10 The C Runtime, Proof Checkers, and Verification Boundaries

The standalone `tcalc` implementation supplies a common C engine for candidate rule firings. Its terms are sorted, interned nodes; its state combines constraint bindings with occurrence identities. Rule records supply take/read premises, equations, disequations, absence/count queries and conclusions. Named-space and birth operations are additional runtime facilities. These mechanisms are candidate implementations of the mathematical interfaces; a formal whole-runtime refinement of `Interface.Step` is not yet established.

10.1 Preparation, admission and the host boundary

`TcPreparedStep` contains a rule identifier, borrowed take/read and instantiation arrays, an optional destination and reachability seeds. `TcCandidateCursor` proposes candidates through `bool (*next)(void *ctx, TcPreparedStep *out)`. The separate bridge computes a `TcPreparedEffect`, checks an explicit `TcAdmissionProfile`, and emits a `TcAdmittedEvent` and `TcAdmissionReceipt` on successful execution.

A cursor need not be trusted for the validity of a candidate that is fully rechecked. Completeness, absence and counting nevertheless require qualified coverage or independent kernel checks. An exhausted budget is not a proof of refutation. Retained events require immutable, live payloads and profile data; the C interface currently borrows these objects. Its policy callbacks must be pure and calls must be externally serialized. A live CeTTa cursor integration and its state-correspondence theorem remain separate obligations.

10.2 Proof-system implementations and their residual trust

Implementation	Input	Evidence and qualification
Micromath	rule operations	Differential substitution/merge/lookup tests; these are not verification of Metamath's `set.mm`.
MM0/MMB	`.mmu`, `.mmb`	Proof and specification fixtures, with malformed and false-proof controls.
HOL	HOLR streams	Primitive inference replay and expected sequents; HOLR is the local binary format, not OpenTheory.
Megalodon	`-sexprinfo`	Natural-deduction proof terms, inductive well-formedness/type checks, and declaration/import rules.
Dedukti	`.dk`	Typing and conversion in the selected LF/CoC and beta/eta profile, compared with named reference checkers.

Protected theorem/signature spaces prevent ordinary external posting after sealing. Conversion, substitution and context operations still rely on named C helpers whose facts enter `SPACE_HELPER`; these routines remain trusted. A ledger names that trust rather than eliminating it. The shell gates require reference executables before reporting reference agreement and distinguish Kontroli's shared-context article checks from separate modules. Kontroli's warnings about unannotated rewrite variables qualify the evidence from those inputs; successful replay alone does not certify their rewrite metatheory.

The November 2026 review added controls for three acceptance gaps: eta was previously enabled silently in the Dedukti checker, an unproved Megalodon `THM` could be closed by `QED`, and a definition identifier could be restated with a different body of the same type. The default conversion now excludes eta; `-eta` is explicit. Proof articles require a checked proof before `QED`, and incompatible definition restatements are refused.

LCF is a theorem-construction discipline, not a mutable tactic stack. It is compatible with natural-deduction proof terms and retained proof DAGs. The current Megalodon checker uses proof terms without a separate `-nd` mode. Equal counts of eight core proof constructors do not establish a duality between simple and dependent type theories; both implementations also perform formation, conversion, declaration and helper operations.

10.3 Translation and machine-code verification

The Megalodon-to-Dedukti exporter emits an STT encoding with type and proof decoding families. It tracks binding context and definition transparency. Corpus replay and mutated-proof rejection test this implementation. General typing, substitution, conversion, proof-preservation and conservativity statements require their own proofs. Selective opacity is a qualification of the exported artifact, not an unconditional equality-preserving translation for all subsequent uses.

The reported source-Megalodon and proof-replay timings describe different workloads: source checking includes parsing, elaboration and possibly tactics. Removing one expensive file does not create a matched-input kernel benchmark. Aggregate time divided by firing count does not isolate the cost of a firing or prove that conversion work is constant.

`FLAT_FIRE_CORRECT` proves the exact 211-byte reference function on s2n-bignum's x86-64 model, with no new axioms. It covers acceptance, refusal, empty lists, register restoration and the return address under explicit bounds and separation conditions. Scratch is disjoint from the inputs and destination; the destination may overlap inputs because input reads finish first. The excluded stack/return region is 56 bytes, of which 48 are writable. Descriptor offset addition is modulo 2^{32}. A packing adapter using natural-number indices must separately discharge its nonwrapping and bounds obligations. This theorem is not yet a proof of the pointer-based tcalc adapter, the C checkers, their parsers, or arbitrary compiler output.

The authored transform presentation and OSLF development provide a route to qualifying those implementations. Preservation of modalities, trace quotients and source proof judgments across the exported tower is a remaining integration goal, not a consequence of sharing the same C engine.

11 Conclusion and Mechanization Summary

The Transform Calculus supplies mathematical interfaces for constrained multiset transitions, knowledge transport, runs and observation. The Lean results cited by declaration name establish their stated hypotheses and conclusions. The C proof-system suite supplies executable examples and regression evidence; the HOL Light leaf proofs establish a separate machine-code boundary. Connecting the full authored presentation, the C implementations and those machine-code results remains essential work.

References

[1] A. Abel and B. Pientka. Higher-order dynamic pattern unification for dependent types and records. In *Typed Lambda Calculi and Applications (TLCA)*, LNCS 6690, pp. 10–26. Springer, 2011.

[2] J.-M. Andreoli and R. Pareschi. Linear objects: Logical processes with built-in inheritance. *New Generation Computing*, 9(3–4):445–473, 1991.

[3] J.-P. Banâtre and D. Le Métayer. The Gamma model and its discipline of programming. *Science of Computer Programming*, 15(1):55–77, 1990.

[4] H. P. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pp. 117–309. Oxford University Press, 1992.

[5] G. Berry and G. Boudol. The chemical abstract machine. *Theoretical Computer Science*, 96(1):217–248, 1992.

[6] C. E. Brown. Megalodon: Proof checking and automated proving for higher-order Tarski–Grothendieck set theory. System documentation and formal library, Czech Technical University in Prague, 2019–2025.

[7] M. Carneiro. Metamath Zero: Designing a theorem prover prover. In *Intelligent Computer Mathematics (CICM)*, LNCS 12236, pp. 71–88. Springer, 2020.

[8] I. Cervesato, N. A. Durgin, P. D. Lincoln, J. C. Mitchell, and A. Scedrov. Relating strands and multiset rewriting for security protocol analysis. In *IEEE Computer Security Foundations Workshop (CSFW)*, pp. 35–51, 2000.

[9] I. Cervesato, F. Pfenning, D. Walker, and K. Watkins. A concurrent logical framework I: Judgments and properties. Technical Report CMU-CS-02-101, Carnegie Mellon University, 2002.

[10] D. Cousineau and G. Dowek. Embedding pure type systems in the lambda-Pi-calculus modulo. In *Typed Lambda Calculi and Applications (TLCA)*, LNCS 4583, pp. 102–117. Springer, 2007.

[11] E. Eder. Properties of substitutions and unifications. *Journal of Symbolic Computation*, 1(1):31–46, 1985.

[12] M. Fitting. *First-Order Logic and Automated Theorem Proving*. Springer, 2nd edition, 1996.

[13] T. Frühwirth. *Constraint Handling Rules*. Cambridge University Press, 2009.

[14] J. Gallier, P. Narendran, D. Plaisted, and S. Raatz. Rigid E-unification is NP-complete. *Information and Computation*, 87(1–2):129–195, 1992.

[15] B. Goertzel, S. Bogdanov, A. Potapov, et al. OpenCog Hyperon and the MeTTa language: Architecture and specification. OpenCog Foundation Technical Reports, 2023–2026.

[16] R. Hähnle. Tableaux and related methods. In A. Robinson and A. Voronkov, editors, *Handbook of Automated Reasoning*, volume I, chapter 3, pp. 100–178. Elsevier, 2001.

[17] J. Harrison. HOL Light: An overview. In *Theorem Proving in Higher Order Logics (TPHOLs)*, LNCS 5674, pp. 60–66. Springer, 2009.

[18] J. S. Hodas and D. Miller. Logic programming in a fragment of intuitionistic linear logic. *Information and Computation*, 110(2):327–365, 1994.

[19] J. Jaffar and M. J. Maher. Constraint logic programming: A survey. *Journal of Logic Programming*, 19/20:503–581, 1994.

[20] D. E. Knuth and P. B. Bendix. Simple word problems in universal algebras. In J. Leech, editor, *Computational Problems in Abstract Algebra*, pp. 263–297. Pergamon Press, 1970.

[21] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[22] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.

[23] D. Miller and G. Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

[24] R. Milner. *A Calculus of Communicating Systems*, LNCS 92. Springer, 1980.

[25] G. D. Plotkin. Call-by-name, call-by-value and the λ-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.

[26] P. Van Weert, J. Sneyers, T. Schrijvers, and B. Demoen. Extending CHR with negation as absence. In *Workshop on Constraint Handling Rules (CHR)*, pp. 125–140, 2006.